



Instituto Mauá de Tecnologia

Curso: Engenharia da Computação

Disciplina: Linguagem de Programação I

Divisão: [preencher divisão]

Sistema Desktop para Controle de Figurinhas da Copa do Mundo 2026

Ruy D. Monteiro - 24.00846-0

Amanda Bialer - 24.01621-7

São Caetano do Sul

2 de junho de 2026

1 Resumo

Este artigo apresenta o desenvolvimento de um aplicativo desktop para acompanhamento de álbuns de figurinhas da Copa do Mundo FIFA 2026. O sistema resolve um problema comum entre colecionadores: identificar quais figurinhas já foram obtidas, quais ainda faltam e quais estão repetidas para troca. A aplicação foi desenvolvida em Java, com interface gráfica construída manualmente em Swing, persistência em MySQL e acesso aos dados por JDBC. O ambiente de banco foi isolado com Docker, permitindo que o projeto seja reproduzido em diferentes computadores sem instalação manual do servidor. A arquitetura adotada separa domínio, repositórios, infraestrutura, configuração e interface gráfica, reduzindo dependências entre as partes e mantendo as responsabilidades claras. Também foram aplicados componentes reutilizáveis, seções de tela, tema visual centralizado, carregamento assíncrono com `SwingWorker`, filtros, busca e métricas de acompanhamento da coleção. Como resultado, foi entregue uma aplicação funcional com tela inicial de progresso, álbum completo, controle de quantidade por figurinha e indicadores de figurinhas coletadas, faltantes e repetidas.

Palavras-chave: Java. Swing. MySQL. JDBC. Álbum de figurinhas. Docker.

Sumário

1	Resumo	1
2	Introdução	3
3	Banco de Dados	5
4	Ambiente	7
5	Estrutura de Arquivos e Arquitetura	8
6	Interface Gráfica	10
7	Desafios Técnicos	13
7.1	WrapLayout: grid com quebra de linha	13
7.2	SwingWorker e responsividade	14
7.3	Atualização local e percepção de desempenho	14
7.4	Quantidade como estado único	14
7.5	Separação entre migrations e seed	15
7.6	Interface em Swing sem gerador visual	15
8	Conclusão	16
	Bibliografia	17

2 Introdução

Este artigo técnico apresenta o desenvolvimento de um aplicativo desktop voltado ao acompanhamento de álbuns de figurinhas da Copa do Mundo FIFA 2026. O projeto parte de uma situação comum no contexto brasileiro: a coleção de figurinhas cresce rapidamente, as páginas passam a ter centenas de itens e a organização manual deixa de ser suficiente para acompanhar faltantes e repetidas.

O problema central do sistema é oferecer uma forma simples de responder três perguntas: quais figurinhas o usuário possui, quais ainda faltam e quais podem ser separadas para troca. Sem uma ferramenta de apoio, essas informações costumam ficar espalhadas em anotações, mensagens ou conferências manuais no próprio álbum, o que aumenta a chance de erro e dificulta a troca entre colecionadores.

Para isso, foi desenvolvida uma aplicação Java com interface Swing, banco de dados MySQL, acesso por JDBC e ambiente de banco provisionado com Docker. A escolha por Swing mantém o projeto dentro do escopo de uma aplicação desktop tradicional, enquanto o uso de JDBC deixa explícito o caminho entre as ações da interface e as operações SQL executadas no banco, conforme a documentação da plataforma Java (Oracle, 2026a) e do MySQL Connector/J (Oracle, 2026b).

Os integrantes do grupo já tinham contato com outras linguagens, bibliotecas e frameworks antes do projeto. Mesmo assim, muitos recursos específicos do ecossistema Java não eram conhecidos no início. A experiência anterior ajudou principalmente a reconhecer padrões úteis e buscar equivalentes em Java: camada de dados, componentes visuais reutilizáveis, arquivos de configuração, separação por seções de tela e isolamento de infraestrutura. O desenvolvimento, portanto, não foi apenas uma implementação de requisitos, mas também uma tradução de conceitos já conhecidos para um ambiente novo.

A solução inclui uma tela inicial com resumo do progresso da coleção, uma tela de álbum com todas as seções e figurinhas, filtros por situação, busca por código ou nome, controle de quantidade e indicadores de repetidas. O escopo foi mantido propositalmente local e individual: a versão atual não possui login, sincronização entre computadores, múltiplos

usuários ou negociação automática de trocas. Essas possibilidades são extensões naturais, mas não são necessárias para demonstrar as decisões de engenharia adotadas no projeto.

3 Banco de Dados

O banco de dados foi modelado para representar o álbum de forma clara e extensível. A decisão principal foi não tratar a Copa de 2026 como um caso especial amarrado ao código. Mesmo que a interface atual exiba apenas esse álbum, a estrutura foi pensada para permitir outros álbuns no futuro, inclusive outras Copas, sem refazer a modelagem central.

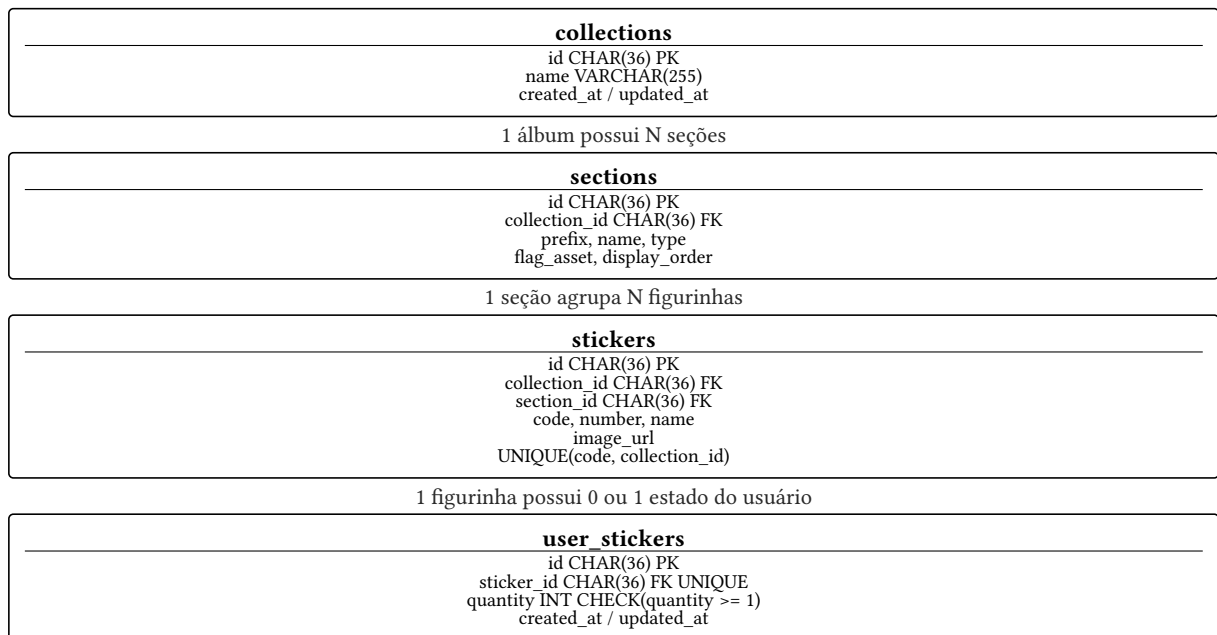


Figura 1: Modelagem relacional adotada para álbuns, seções, figurinhas e estado da coleção.

A tabela `collections` existe mesmo com apenas um álbum em uso, pois o custo dessa generalização é baixo e evita que toda a estrutura fique presa à Copa de 2026. Se outro álbum for cadastrado, como uma Copa futura, a nova coleção passa a reutilizar as mesmas tabelas de seções, figurinhas e progresso do usuário. Essa granularidade mantém o sistema simples no uso atual, mas não fecha portas para expansão.

O campo `code` da figurinha também não foi usado como chave primária. Códigos como `BRA1`, `FWC1` ou `MEX14` são identificadores visuais do álbum, não identificadores globais do sistema. Se dois álbuns existirem em paralelo, o código `BRA1` pode aparecer em mais de uma coleção. Por isso, cada figurinha possui um `id` próprio em formato `UUID`, enquanto `code` é único apenas em conjunto com `collection_id`. Essa escolha evita colisões futuras sem sacrificar a busca por código na interface.

A tabela `sections` abstrai as divisões do álbum. Ela permite tratar seleções, páginas especiais, figurinhas Panini, histórico da Copa e outros agrupamentos de maneira uniforme. O campo `type` diferencia seções do tipo `team`, `special` e `regional`, enquanto `display_order` controla

a ordem de exibição. Assim, a interface não precisa saber se uma seção representa uma seleção ou uma página especial; ela apenas renderiza grupos ordenados com suas respectivas figurinhas.

A tabela `user_stickers` representa o estado da coleção do usuário. A ausência de registro indica que a figurinha não foi coletada. Um registro com `quantity = 1` indica posse, e valores maiores que um indicam figurinhas repetidas para troca. Essa modelagem evita campos booleanos redundantes, como `collected` ou `repeated`, porque essas informações podem ser derivadas diretamente da quantidade. A restrição `quantity >= 1` mantém a integridade no banco e reforça essa regra: quantidade zero não é um estado persistido, mas a ausência do registro.

Os arquivos SQL foram separados em `db/migrations` e `db/seed`. As migrations definem a estrutura do banco, como tabelas, chaves, índices e restrições. Os seeds inserem dados iniciais, como coleção, seções e figurinhas. Essa separação deixa claro o que é estrutura e o que é dado de carga inicial, além de permitir recriar o ambiente de forma previsível. A numeração sequencial dos arquivos, como 01, 02 e 03, também foi adotada por simplicidade: a ordem de execução fica explícita e não depende de interpretação por data, timestamp ou ferramenta externa.

O seed principal cobre o recorte usado pelo projeto, com 52 seções e 1034 figurinhas. A coleta e conferência dos dados passou por mais de uma fonte. Parte das informações foi levantada no LastSticker, que mantém página de checklist da coleção Panini FIFA World Cup 2026 (LastSticker, 2026). Depois, a lista de seleções e dados de times também foi comparada com o arquivo aberto `worldcup.teams.json` do projeto OpenFootball (OpenFootball, 2026). O resultado foi tratado como dado inicial do sistema, não como regra de negócio fixa.

A construção do seed também exigiu tratamento de qualidade de dados. Alguns nomes de figurinhas apresentaram caracteres incorretos por problema de codificação, e os arquivos de bandeiras precisaram ser comparados com os códigos ISO usados nas seções. Em outros momentos, foi necessário decidir se uma informação deveria ficar no banco, como o código FIFA, ou como recurso visual da interface, como o nome do asset de bandeira. Essa etapa mostrou que seed não é apenas inserir dados no banco: também envolve conferência, normalização e decisões sobre a fronteira entre dado persistido e recurso da aplicação.

4 Ambiente

O banco de dados do projeto roda em MySQL por meio de Docker. Essa decisão melhora a reprodutibilidade: qualquer integrante do grupo consegue subir o mesmo ambiente de banco sem instalar e configurar o MySQL manualmente no computador. O Docker Compose concentra imagem, porta, volume e credenciais em um único arquivo, seguindo a proposta de isolamento descrita pela documentação do Docker (Docker Inc., 2026).

O aplicativo Java não foi containerizado. Como a interface foi feita em Swing, ela depende do sistema operacional para exibir janelas. Executar uma aplicação gráfica desktop dentro de um container exigiria configurações extras, como encaminhamento de display, sem trazer benefício real para o escopo do projeto.

O volume do Docker mantém os dados do MySQL entre reinicializações. Assim, o banco não é descartado toda vez que o container é desligado, preservando o comportamento esperado de uma aplicação com persistência não volátil. Quando o ambiente precisa ser recriado do zero, a separação entre migrations e seed permite apagar o volume, subir o serviço e reaplicar a estrutura e os dados iniciais em ordem.

O acesso ao banco foi feito com JDBC explícito, sem ORM. Essa escolha atende diretamente ao requisito da disciplina e também torna as consultas visíveis. O fluxo entre Java e SQL pode ser acompanhado sem camadas de abstração escondidas. A classe `DatabaseConnection` centraliza a conexão, usa o driver `com.mysql.cj.jdbc.Driver` e permite configurar host, porta, banco, usuário e senha por variáveis de ambiente, mantendo valores padrão para execução local.

Mesmo com valores padrão simples, como usuário e senha `root` para o ambiente local, o projeto também documenta as variáveis esperadas em um arquivo `.env.example`. Essa decisão ajuda outros integrantes a configurar o banco sem precisar procurar nomes de variáveis diretamente no código e evita que credenciais reais sejam versionadas por acidente.

Para evitar que o código de interface ou de domínio conheça detalhes de SQL, foi adotado o padrão Repository. Cada repositório encapsula operações de leitura e escrita relacionadas a uma entidade, mantendo o restante da aplicação mais limpo e organizado.

5 Estrutura de Arquivos e Arquitetura

O projeto foi organizado em camadas, com responsabilidades bem separadas. A pasta `domain` contém as classes que representam conceitos do sistema, como coleção, seção, figurinha, progresso e figurinha do usuário. Essas classes não dependem de Swing, JDBC ou MySQL, o que mantém o domínio simples e portátil.

A camada `data` contém os repositórios. Eles são responsáveis por transformar operações do sistema em consultas SQL. Métodos como `findByCollection`, `findRepeated`, `updateQuantity` e `findProgress` seguem uma nomenclatura previsível, facilitando a leitura do código e evitando mistura de verbos com o mesmo significado.

A pasta `infra` concentra detalhes de infraestrutura, como a conexão com o banco de dados. Essa separação impede que cada classe crie sua própria conexão ou conheça diretamente o driver JDBC. Já a pasta `config` guarda constantes de aplicação, navegação, coleção padrão e regras simples de cálculo, como quantidade de figurinhas por envelope e preço estimado do envelope. Essa separação evita que números de configuração fiquem espalhados pelas telas.

A camada `ui` contém a interface gráfica, dividida em componentes reutilizáveis e telas completas. As telas maiores seguem uma divisão interna entre composição visual e carregamento de dados. `HomeDataLoader` e `AlbumDataLoader` encapsulam as consultas executadas em segundo plano, enquanto `HomeScreen` e `AlbumScreen` coordenam a renderização e os estados de tela. Essa organização evita que a montagem visual fique misturada com a criação de repositórios e chamadas ao banco.

Também foi criada uma divisão por seções dentro das telas. A Home é composta por `HeaderSection`, `CollectionProgressSection`, `HomeStatsSection`, `TeamsProgressSection` e `ActivitySection`. A tela de álbum separa cabeçalho, barra de ferramentas e grade em `AlbumHeaderSection`, `AlbumToolbarSection` e `AlbumGridSection`. Essa escolha impede que cada tela vire uma classe gigante com toda a interface no mesmo arquivo. A tela principal continua responsável por orquestrar estado, filtros e carregamento, enquanto cada seção cuida de uma parte visual coesa.

No banco de dados foi usada a convenção `snake_case`, comum em SQL. No Java foram usadas classes em `PascalCase`, atributos e métodos em `camelCase` e constantes em

UPPER_SNAKE_CASE. Essa decisão respeita o padrão de cada linguagem e reduz estranhamento para quem lê o código.

O projeto não utiliza Maven ou Gradle. As dependências externas ficam na pasta lib, incluindo o FlatLaf e o MySQL Connector/J. Essa escolha torna a compilação mais manual, mas também explícita: o classpath informa diretamente quais bibliotecas são necessárias para executar a aplicação.

6 Interface Gráfica

A interface gráfica foi desenvolvida manualmente com Swing, sem geradores automáticos de código. Isso atende ao requisito do projeto e garante que a construção dos componentes, eventos, layouts e estados visuais seja compreendida pelo grupo. O FlatLaf foi usado como base visual moderna para Swing (FormDev Software GmbH, 2026), e a configuração global da aplicação é aplicada antes da criação da janela principal.

O arquivo Theme.java centraliza o sistema visual do aplicativo. Nele ficam cores, fontes, tamanhos, espaçamentos, raios de borda e dimensões compartilhadas. A versão atual utiliza uma paleta clara, com fundo principal branco, superfícies secundárias em cinza claro e verde #22C55E como cor de destaque. Essa centralização evita valores espalhados pela aplicação e permite alterar a identidade visual em um ponto único.

As fontes Geist são carregadas a partir do classpath da aplicação. Com isso, a aparência da interface não depende das fontes instaladas no computador do usuário. A aplicação tende a manter a mesma leitura visual em diferentes máquinas, preservando a identidade gráfica do projeto.

Os componentes foram pensados de forma atômica. RoundedButton e RoundedPanel não possuem regra de negócio e podem ser reutilizados em qualquer tela. ProgressBarCustom, Badge, StatCard, PageHeader, FilterBar, RoundedTextField, FlagBadge e StickerCard concentram padrões visuais recorrentes. As telas, por sua vez, compõem esses componentes e coordenam os estados principais.

Na tela inicial, o sistema apresenta o progresso geral da coleção, estatísticas de coletadas, faltantes e repetidas, estimativa de envelopes, gasto estimado, progresso por times e atividade recente. A Home foi organizada como um painel de acompanhamento, não apenas como uma lista de figurinhas. Na tela de álbum, o usuário visualiza as seções ordenadas, busca por código ou nome, filtra por status e altera a quantidade de cada figurinha por meio de um diálogo próprio. Essa separação aproxima a Home de uma visão gerencial e o Álbum de uma área de operação.

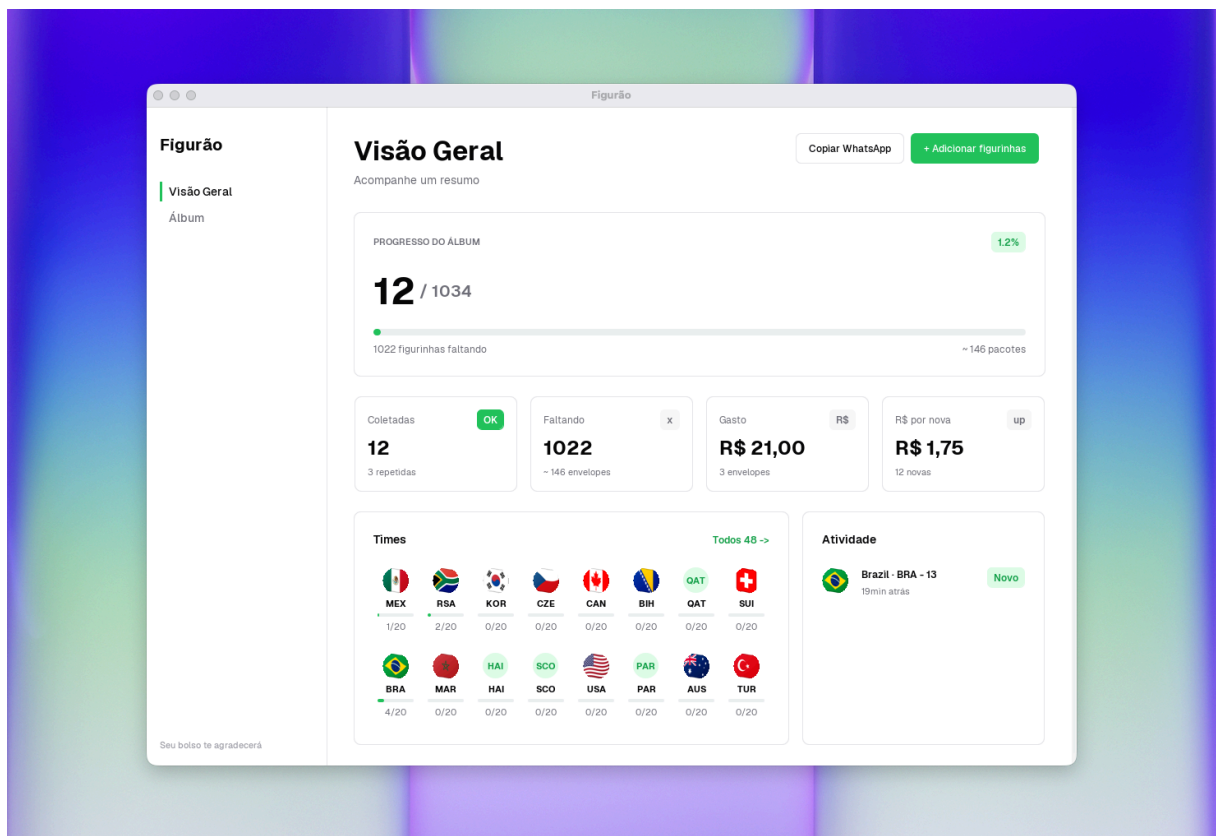


Figura 2: Tela Visão Geral com progresso do álbum, métricas, times e atividade recente.

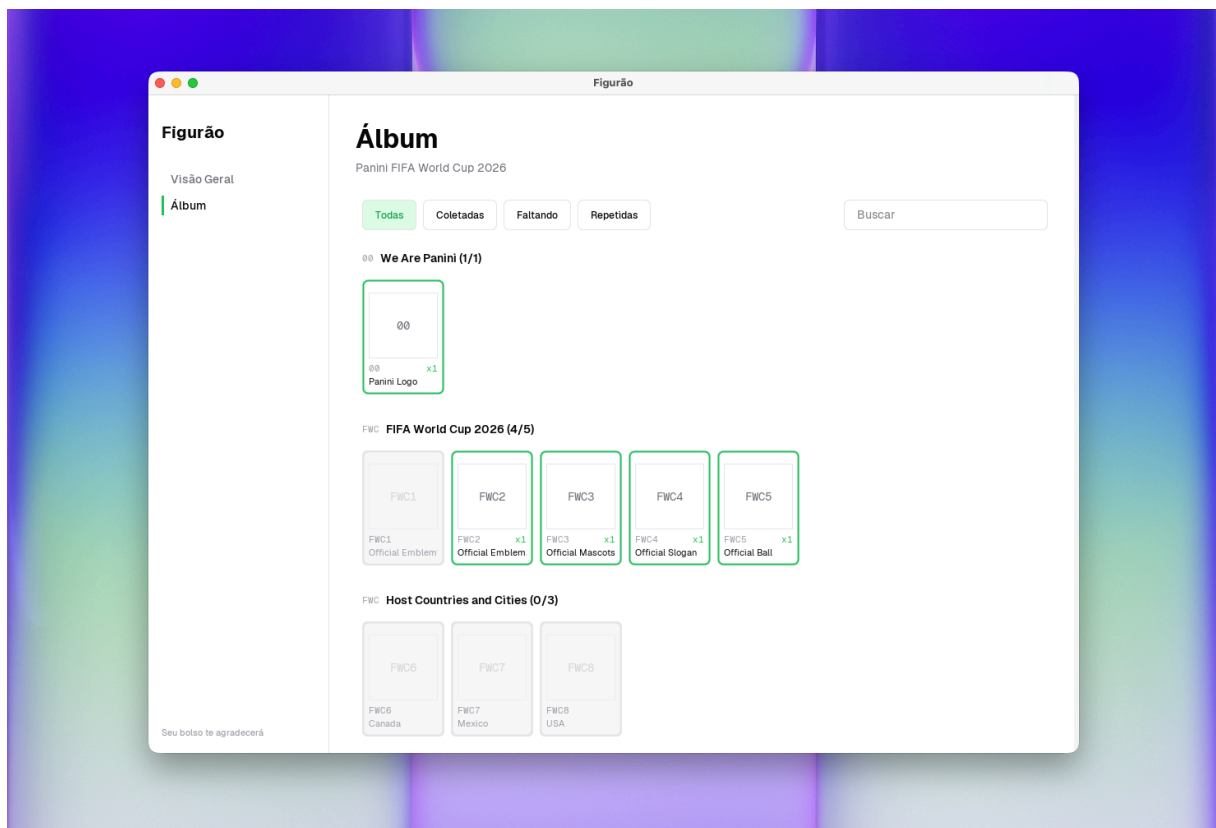


Figura 3: Tela Álbum com filtros por status, busca e grade de figurinhas agrupadas por seção.

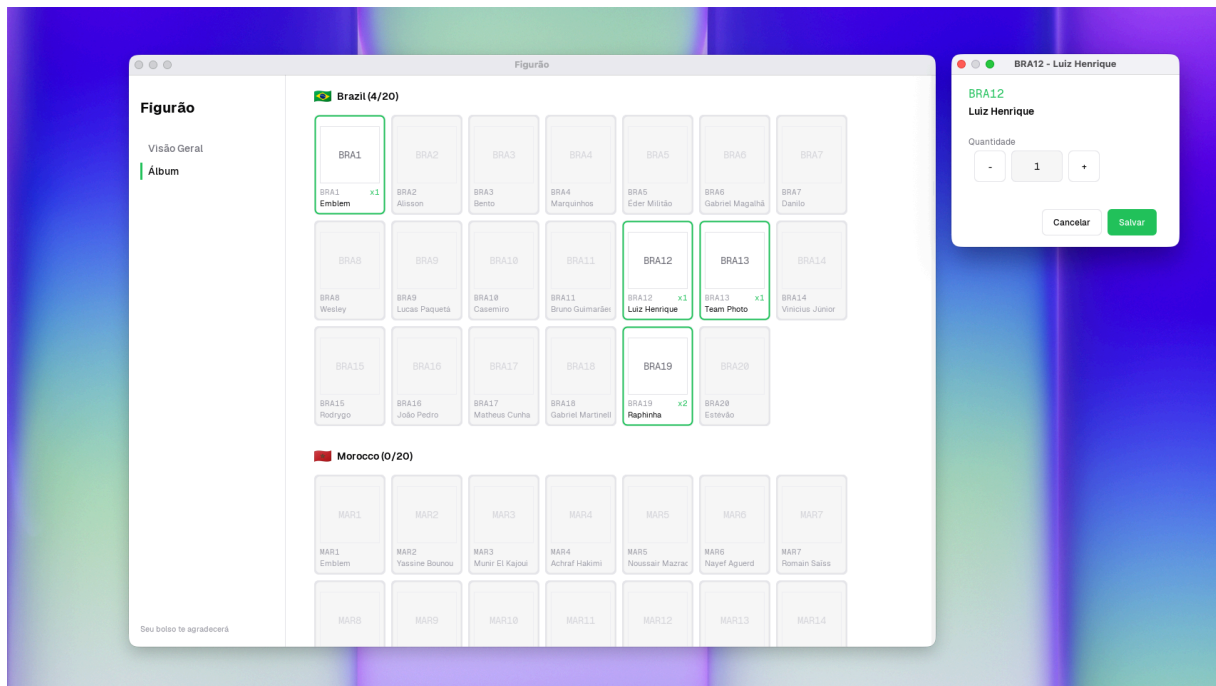


Figura 4: Diálogo de quantidade aberto sobre a tela do Álbum, com controles de incremento e decremento. A divisão entre sections e components foi essencial para controlar o crescimento da interface. Os componentes compartilhados resolvem problemas repetidos de layout e estilo, como cartões, botões, badges, campos e barras de progresso. Já as seções organizam partes maiores da tela, como cabeçalho, barra de filtros, grade do álbum, estatísticas e atividade. Assim, uma mudança visual em um botão ou card não exige editar cada tela, e uma mudança de composição de tela não altera a implementação interna dos componentes.

Elementos customizados, como bordas arredondadas, cartões, barras de progresso e opacidade de figurinhas não coletadas, foram desenhados com `paintComponent()` e `Graphics2D`. Dessa forma, a interface preserva uma aparência mais cuidada sem depender de ferramentas visuais de geração automática.

7 Desafios Técnicos

O projeto poderia ter sido muito mais simples. Uma implementação mínima em Swing, com poucas classes e consultas diretas ao banco, provavelmente atenderia parte dos requisitos. No entanto, o grupo se empolgou durante o desenvolvimento e passou a tentar uma solução progressivamente mais ambiciosa: primeiro uma estrutura de pastas mais organizada, depois uma arquitetura em camadas, em seguida uma interface visualmente mais cuidada e, por fim, um painel com métricas e estados de uso.

Muito do que foi implementado não era conhecido previamente pelos integrantes no ecossistema Java. Ainda assim, a experiência com outras linguagens e frameworks ajudou a formular boas perguntas: como separar dados e interface, como criar componentes reaproveitáveis, como evitar que uma tela vire um arquivo enorme, como carregar dados sem travar a UI e como reproduzir no Swing alguns comportamentos já vistos em outras tecnologias. O desenvolvimento acabou sendo guiado por pesquisa, leitura de documentação, buscas pontuais e uso de agentes para encontrar equivalentes em Java.

7.1 WrapLayout: grid com quebra de linha

O Swing não possui um layout nativo equivalente ao flexbox do CSS. O FlowLayout padrão organiza componentes em linha, mas não resolve todos os cenários de redimensionamento dentro de um JScrollPane. Durante o desenvolvimento, os cards de figurinha chegaram a transbordar horizontalmente em vez de formar uma grade.

A solução foi usar WrapLayout, uma extensão de FlowLayout baseada no trabalho de Rob Camick (Camick, 2008). O layout recalcula o tamanho preferido considerando a largura disponível, permitindo que os cards quebrem linha. Também foi necessário fazer o painel rolável acompanhar a largura do viewport, pois sem isso o layout acreditava ter espaço horizontal ilimitado.

A motivação para procurar essa solução veio de experiências anteriores com interfaces que reaproveitam células ou reorganizam itens conforme o tamanho da tela, como listas e grades em outros frameworks. O grupo não sabia de início qual seria o equivalente em Java, mas sabia qual comportamento procurar. Esse foi um padrão recorrente do projeto: reconhecer o problema por experiência prévia e pesquisar a solução específica para Swing.

7.2 SwingWorker e responsividade

O Swing renderiza a interface na Event Dispatch Thread. Se uma consulta ao banco for executada diretamente nessa thread, a janela pode travar até a operação terminar. Esse problema aparece com mais força no álbum completo, pois a tela precisa consultar coleção, seções, figurinhas e quantidades do usuário.

Para evitar travamentos, as telas Home e Álbum carregam dados por meio de SwingWorker. A consulta ao banco roda em segundo plano e, quando termina, a interface é atualizada com segurança no método `done()`. A mesma ideia aparece no salvamento da quantidade, fazendo com que a alteração no banco não bloqueie a interação principal.

7.3 Atualização local e percepção de desempenho

Um ponto percebido durante os testes foi que carregar dados em segundo plano não resolve, sozinho, todos os problemas de sensação de lentidão. Inicialmente, ao alterar a quantidade de uma figurinha, o sistema salvava no banco, recarregava os dados do álbum e só depois reconstruía a grade. Mesmo com SwingWorker, a interface parecia atrasada, pois o usuário fechava o diálogo e ainda via o estado antigo por alguns instantes.

A solução foi atualizar primeiro o estado local da tela e redesenhar a grade imediatamente, enquanto o salvamento no MySQL acontece em segundo plano. Caso o salvamento falhe, o estado anterior é restaurado e a tela de erro é exibida. Essa estratégia melhora a percepção de resposta sem abandonar a persistência no banco. O desafio mostrou que desempenho em interface não depende apenas de evitar travamentos, mas também de quando a aplicação comunica visualmente que uma ação foi concluída.

7.4 Quantidade como estado único

Outro desafio foi representar coletadas, faltantes e repetidas sem multiplicar flags no banco. A solução foi tratar a quantidade como estado único. Quando não existe registro em `user_stickers`, a figurinha é faltante. Quando existe registro com quantidade um, ela foi coletada. Quando a quantidade é maior que um, a diferença representa as repetidas disponíveis.

Essa decisão simplifica filtros, progresso e cálculo de métricas. Ao mesmo tempo, exige cuidado no decremento e no diálogo de edição: se o usuário reduz a quantidade para zero, o registro deve ser removido, e não atualizado para um valor inválido.

7.5 Separação entre migrations e seed

Também foi necessário organizar a criação do banco. Misturar estrutura e dados em um único script deixaria o processo menos claro. Por isso, migrations e seeds foram separados. As migrations criam tabelas e restrições, enquanto os seeds inserem a coleção, as seções e as figurinhas iniciais. Essa organização facilita explicar o banco, recriar o ambiente e entender quais arquivos alteram estrutura e quais apenas adicionam dados.

7.6 Interface em Swing sem gerador visual

Construir a interface manualmente foi outro ponto trabalhoso. O Swing permite bastante controle, mas exige lidar diretamente com layouts, tamanhos, bordas, eventos e pintura. Para evitar um código monolítico, o grupo separou elementos pequenos em componentes compartilhados e blocos maiores em seções de tela. Essa separação tornou possível continuar refinando a interface sem transformar HomeScreen e AlbumScreen em classes gigantes.

8 Conclusão

O projeto demonstra que é possível construir um aplicativo desktop funcional e organizado usando Java, Swing, JDBC e MySQL. O sistema atende aos requisitos básicos da disciplina e também aplica decisões de engenharia que tornam o código mais claro e fácil de evoluir.

A separação entre domínio, repositórios, infraestrutura e interface reduziu o acoplamento entre as partes. O uso de Docker tornou o banco reproduzível. O padrão Repository manteve o JDBC controlado e rastreável. O design system em Theme.java e os componentes reutilizáveis trouxeram consistência visual sem complexidade excessiva.

A versão atual possui limitações conscientes. O sistema é local, não possui login, não sincroniza dados entre computadores e considera apenas um usuário. Essas restrições mantêm o projeto adequado ao escopo da disciplina e evitam adicionar dependências que desviariam o foco dos conceitos principais.

Ao mesmo tempo, a aplicação ficou mais ambiciosa do que uma entrega mínima. Esse resultado veio de um processo de pesquisa contínua: a cada iteração, o grupo encontrava um problema novo, procurava como ele era resolvido no universo Java e tentava adaptar a solução ao projeto. A principal aprendizagem não foi apenas escrever classes em Java, mas perceber que padrões já conhecidos em outras tecnologias podem ser transportados, com ajustes, para um ecossistema diferente.

Como evoluções naturais, seria possível adicionar múltiplos usuários, suporte a outros álbuns, sincronização em rede, importação de imagens oficiais e troca de figurinhas entre usuários cadastrados. Mesmo sem essas extensões, a aplicação entregue resolve o problema principal: acompanhar a coleção e facilitar a organização das trocas.

Bibliografia

CAMICK, Rob. **Wrap Layout**. Disponível em: <<https://tips4java.wordpress.com/2008/11/06/wrap-layout/>>. Acesso em: 2 jun. 2026.

DOCKER INC. **Docker Documentation**. Disponível em: <<https://docs.docker.com/>>. Acesso em: 2 jun. 2026.

FORMDEV SOFTWARE GMBH. **FlatLaf: Flat Look and Feel**. Disponível em: <<https://www.formdev.com/flatlaf/>>. Acesso em: 2 jun. 2026.

LASTSTICKER. **Panini FIFA World Cup 2026. Standard Edition**. Disponível em: <https://www.laststicker.com/cards/panini_world_cup_2026/>. Acesso em: 2 jun. 2026.

OPENFOOTBALL. **worldcup.teams.json**. Disponível em: <<https://github.com/openfootball/worldcup.json/blob/master/2026/worldcup.teams.json>>. Acesso em: 2 jun. 2026.

ORACLE. **Java Platform, Standard Edition Documentation**. Disponível em: <<https://docs.oracle.com/en/java/javase/>>. Acesso em: 2 jun. 2026a.

ORACLE. **MySQL Connector/J Developer Guide**. Disponível em: <<https://dev.mysql.com/doc/connector-j/en/>>. Acesso em: 2 jun. 2026b.